

گزارش تمرین اول - پیاده‌سازی الگوریتم DES

محمد امانلو - شماره دانشجویی: ۸۱۰۱۰۰۰۸۴

۱ مقدمه

در این تمرین هدف این بود که الگوریتم رمزنگاری DES را به صورت کامل پیاده‌سازی کنیم. من سعی کردم کل فرآیند را از ابتدا تا انتها خودم درک کنم و بعد قدم به قدم پیاده‌سازی کنم، طوری که همه اجزای مهم مثل تولید کلیدهای دور، تابع Feistel و S-box به درستی کار کنند. در کل، تمرکز اصلی این بود که فقط یک خروجی بگیریم نیست، بلکه بفهمیم داخل DES دقیقا چه اتفاقی می‌افتد.

۲ بررسی کدهای استارتر

در ابتدای کار، یک سری کد استارتر در اختیار ما قرار داده شده بود که خیلی کمک کرد مسیر را بفهمم. به طور خلاصه:

- فایل `hw1_starter.py` ساختار کلی رمزنگاری را نشان می‌داد، مخصوصا اینکه چطور داده به دو نیمه تقسیم می‌شود و چطور هر دور اجرا می‌شود.
- فایل `generate_round_keys.py` نحوه تولید کلیدهای هر دور را نشان می‌داد، یعنی همان `schedule, key`
- فایل `illustrate_des_substitution.py` هم خیلی مهم بود چون دقیقا نحوه کار با S-box را توضیح داده بود.

من اول این فایل‌ها را خط به خط بررسی کردم تا بفهمم هر بخش چه کاری انجام می‌دهد، بعد شروع کردم به نوشتن کد اصلی.

۳ توضیح الگوریتم DES

الگوریتم DES یک الگوریتم بلوکی است که روی بلوک‌های ۶۴ بیتی کار می‌کند و شامل ۱۶ دور Feistel است. روند کلی به این صورت است:

۱. ابتدا روی ورودی یک Initial Permutation انجام می‌شود.

۰۲. داده به دو بخش ۳۲ بیتی چپ و راست تقسیم می‌شود.

۰۳. در هر دور:

- بخش راست وارد تابع F می‌شود
- خروجی آن با بخش چپ XOR می‌شود
- سپس جای دو بخش عوض می‌شود

۰۴. بعد از ۱۶ دور، دو بخش به هم متصل می‌شوند

۰۵. در نهایت Inverse Permutation اعمال می‌شود

تابع F هم شامل سه مرحله اصلی است:

• Expansion (۳۲ بیت به ۴۸ بیت)

• XOR با کلید دور

• عبور از S-box و سپس یک permutation دیگر

۴ چالش نصب کتابخانه

یکی از چالش‌هایی که داشتم نصب کتابخانه BitVector بود. به خاطر محدودیت‌های اینترنتی (و شرایط خاص شبکه)، نصب مستقیم pip خیلی راحت انجام نمی‌شد. در نهایت با اجرای دستور زیر روی همان محیط پایتون که کد را اجرا می‌کردم، مشکل حل شد:

```
BitVector install pip -m python
```

نکته مهم این بود که باید دقیقاً روی همان نسخه پایتون نصب می‌شد که کد با آن اجرا می‌شود.

۵ پیاده‌سازی انجام‌شده

در کدی که نوشتم، کل فرآیند DES را مرحله به مرحله پیاده‌سازی کردم.

۱۰۵ گرفتن ورودی

ابتدا از کاربر دو ورودی گرفتم:

• plaintext (۸ کاراکتر)

• key (۸ کاراکتر)

این‌ها به کمک BitVector به بیت تبدیل شدند.

۲۰.۵ تولید کلیدهای دور

کلید اولیه با استفاده از permutation مربوطه (PC-۱) به ۵۶ بیت تبدیل شد. بعد به دو بخش تقسیم شد و در هر دور شیفیت داده شد و در نهایت با PC-۲ کلید هر دور ساخته شد.

۳۰.۵ اجرای ۱۶ دور Feistel

در هر دور:

- بخش راست expand شد
- با کلید XOR شد
- وارد ها S-box شد
- permutation انجام شد
- با بخش چپ XOR شد
- و در نهایت دو بخش جابجا شدند.

۴۰.۵ S-box

برای ها S-box دقیقاً از ساختار استاندارد استفاده کردم. هر ۶ بیت به یک S-box رفت و خروجی ۴ بیت تولید شد.

۶ توضیح جزئی تر که

در این بخش سعی می‌کنم دقیق‌تر توضیح بدهم که داخل کدی که نوشتم چه اتفاقی می‌افتد و هر قسمت چه کاری انجام می‌دهد.

۱۰.۶ ساختار کلی برنامه

کد به صورت ماژولار نوشته شده، یعنی هر قسمت از الگوریتم در یک تابع جدا پیاده‌سازی شده است. این کار باعث شد هم خوانایی بهتر شود و هم بتوانم هر بخش را جدا تست کنم. توابع اصلی که نوشتم شامل موارد زیر هستند:

- تابع تولید کلیدهای دور
- تابع Feistel
- تابع اصلی رمزنگاری DES

۲.۶ تابع تولید کلیدهای دور

در این قسمت، ابتدا کلید ورودی که ۸ کاراکتر است، به BitVector تبدیل می‌شود. بعد با استفاده از جدول PC-۱، ۶۴ بیت به ۵۶ بیت تبدیل می‌شود. بعد کلید به دو بخش چپ و راست تقسیم می‌شود. در هر دور:

- هر دو بخش به اندازه مشخصی شیفت داده می‌شوند
- دوباره به هم چسبانده می‌شوند
- با استفاده از PC-۲ به ۴۸ بیت تبدیل می‌شوند
- در نهایت ۱۶ کلید مختلف برای ۱۶ دور تولید می‌شود.

۳.۶ تابع Feistel

این تابع مهم‌ترین قسمت الگوریتم است. اول بخش راست (۳۲ بیت) را با استفاده از جدول Expansion به ۴۸ بیت تبدیل کردم. بعد آن را با کلید همان دور XOR کردم. خروجی به ۸ بخش ۶ بیتی تقسیم شد و هر بخش وارد یک S-box شد. در هر S-box:

- دو بیت اول و آخر شماره سطر را مشخص می‌کنند
- چهار بیت وسط شماره ستون را مشخص می‌کنند
- مقدار خروجی از جدول خوانده می‌شود

در نهایت خروجی ها S-box که ۳۲ بیت است، یک permutation دیگر روی آن انجام می‌شود.

۴.۶ تابع اصلی DES

در این تابع، ابتدا plaintext به BitVector تبدیل شد. بعد جایگشت اولیه (IP) روی آن انجام شد. سپس داده به دو قسمت تقسیم شد:

- L (چپ)
- R (راست)

در هر یک از ۱۶ دور:

- مقدار R موقت ذخیره شد
- مقدار جدید R برابر شد با XOR بین L و خروجی تابع Feistel
- مقدار L برابر مقدار قبلی R شد

در انتهای ۱۶ دور، دو بخش جابجا شده و به هم متصل شدند. در نهایت جایگشت معکوس (IP^{-1}) روی کل داده اعمال شد و خروجی نهایی تولید شد.

۵.۶ نمایش خروجی

در انتها خروجی به صورت HEX نمایش داده شد تا خوانا تر باشد و بتوان راحت تر مقایسه انجام داد.

۶.۶ نکات مهم در پیاده سازی

چند نکته ای که هنگام پیاده سازی خیلی مهم بود:

- دقت در اندیس ها (چون جداول از ۱ شروع می شوند ولی پایتون از ۰)
 - درست تقسیم کردن بیت ها در S-box
 - رعایت ترتیب دقیق مراحل الگوریتم
- اگر یکی از این ها اشتباه می شد، کل خروجی اشتباه می شد.

۷ نتایج اجرا

برای اطمینان از صحت کد، چند نمونه تست اجرا کردم:

۱.۷ تست اول

ABCDEFGH plaintext:
12345678 key:

خروجی:

96de603eae6256f

۲.۷ تست دوم

HELLOWRD plaintext:
SECRETS! key:

خروجی:

5f3acde2a0c31b35

۸ بررسی صحت نتایج

برای بررسی درست بودن کد چند نکته را چک کردم:

- برای ورودی یکسان، خروجی همیشه یکسان بود (deterministic بودن)
- با تغییر ورودی، خروجی کاملاً تغییر می کرد

- چند بار اجرای پشت سر هم، نتایج ثابت باقی ماندند
- این‌ها نشان می‌دهد که الگوریتم به درستی پیاده‌سازی شده است.

۹ اسکرین‌شات نتایج

در این بخش، تصویر اجرای برنامه در محیط ترمینال قرار داده شده است:

```
PS C:\Users\A\Desktop\HW1_starter_code> & C:/Users/a/anaconda3/2/python.exe c:/Users/a/Desktop/HW1_starter_code/q1_des.py
Enter plaintext (8 chars): ABCDEFGH
Enter key (8 chars): 12345678

Cipher (HEX): 96de603eae6256f
PS C:\Users\A\Desktop\HW1_starter_code> & C:/Users/a/anaconda3/2/python.exe c:/Users/a/Desktop/HW1_starter_code/q1_des.py
Enter plaintext (8 chars): HELLOWRD
Enter key (8 chars): SECRETS!

Cipher (HEX): 5f3acde2a0c31b35
PS C:\Users\A\Desktop\HW1_starter_code> & C:/Users/a/anaconda3/2/python.exe c:/Users/a/Desktop/HW1_starter_code/q1_des.py
Enter plaintext (8 chars): HELLOWRD
Enter key (8 chars): SECRETS!

Cipher (HEX): 5f3acde2a0c31b35
PS C:\Users\A\Desktop\HW1_starter_code> & C:/Users/a/anaconda3/2/python.exe c:/Users/a/Desktop/HW1_starter_code/q1_des.py
Enter plaintext (8 chars): ABCDEFGH
Enter key (8 chars): 12345678

Cipher (HEX): 96de603eae6256f
```